

75. Le fichier voisin écrit ses propres #include

<...>

utils.cpp se sert de poser() : c'est donc lui qui écrit #include <poser>. Le fichier se suffit à lui-même.

1. Ce qu'il faut comprendre

Le seul changement : la ligne #include <poser> a changé d'onglet. Elle était dans principal.cpp ; elle est maintenant en haut d'utils.cpp.

Pourquoi c'est mieux : c'est bande() qui se sert de poser(), pas main(). En écrivant la ligne **dans le fichier qui en a besoin**, utils.cpp se suffit à lui-même : un autre programme qui écrit #include "utils.cpp" n'a rien d'autre à penser.

Et si deux fichiers écrivent la même ligne ? Si principal.cpp gardait aussi son #include <poser>, ce ne serait pas une erreur : une fonction de la console n'est gravée **qu'une fois**, et une ligne de trop ne coûte rien.

Le texte versé, en entier : #include <ALPHABET>, puis tout utils.cpp (son #include <poser> et bande()), puis main(). Un #include <...> vaut pour **tout** le programme assemblé, où qu'il soit écrit : celui d'utils.cpp autorise poser() partout.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **949 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	49 o	← la console, à l'allumage — la mise en route de la console
bande()	38 o	← main() — une fonction fournie par le langage
main()	64 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AdresseCase	10 o	← poser() — trouve l'adresse d'une case de l'écran
CopierTuiles	18 o	← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage — vide les deux cartes de l'écran
Tuiles	704 o	← CopierTuiles — la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreAcces, AttendreFinVBlank, EffacerCases, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

poser()	× 1	17 o sur place, dans bande() + AdresseCase (10 o)
image()	× 1	3 o sur place, dans main() + AttendreImage (28 o)

Tes fonctions appelées :

bande()	× 3	57 o sur place, dans main()
---------	-----	-----------------------------

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

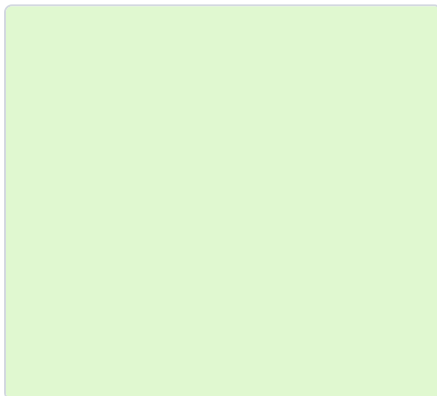
```
// Le changement : « #include <poser> » est parti dans outils.cpp,  
// le fichier qui s'en sert. main() n'appelle pas poser() lui-même.  
  
#include <ALPHABET> // les lettres : c'est main() qui s'en sert  
#include "outils.cpp" // mes fonctions (et leurs #include à elles)
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

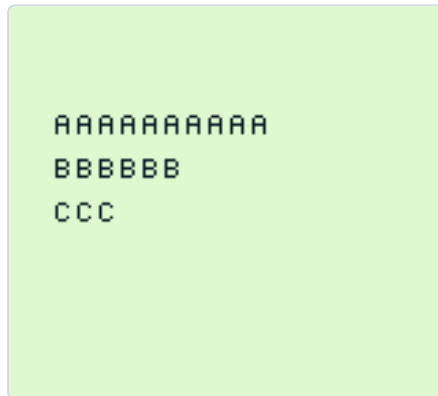
```
int main() {  
    bande(2, 5, ALPHABET[0], 10);  
    bande(2, 7, ALPHABET[1], 6);  
    bande(2, 9, ALPHABET[2], 3);  
  
    while (true) {  
        image();  
    }  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Toujours le même écran : dix A, six B, trois C.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ outils.cpp inclut poser() lui-même : les bandes sont là

